

Java Basics – Command Line Arguments, Prime Number and Palindrome

Experiment No: 03

Date: ____ / ____ / 2026

Title: Command Line Arguments, Prime Number and Palindrome Checking using Methods in Java

■ Objective / Aim

To write a Java program that demonstrates Command Line Arguments, checks whether a given number is Prime or not, and checks whether a given number is a Palindrome or not using methods.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

Steps / Procedure

1. Start the program.
2. Import `java.util.Scanner`.
3. Create a class named **JavaBasics**.
4. Create the `isPrime()` method to check whether a number is prime.
5. Create the `isPalindrome()` method to check whether a number is a palindrome.
6. In `main()`, display the command-line arguments.
7. Read a number using `Scanner`.
8. Call `isPrime()` and display the result.
9. Read another number.
10. Call `isPalindrome()` and display the result.
11. Close the `Scanner`.
12. Stop the program.

Prime Number Algorithm

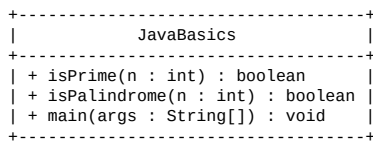
1. Read integer `n`.

2. If $n \leq 1$, return false.
3. Set $i = 2$.
4. Repeat while $i \leq n/2$.
5. If $n \% i == 0$, return false.
6. Increment i .
7. If no divisor is found, return true.

Palindrome Algorithm

1. Read integer n .
2. Store the original value in original.
3. Set reverse = 0.
4. Extract the last digit using $n \% 10$.
5. Add the digit to reverse.
6. Remove the last digit using $n / 10$.
7. Repeat until $n = 0$.
8. Compare original with reverse.
9. If equal, the number is a palindrome.
10. Otherwise, it is not a palindrome.

■ UML Class Diagram



■ Source Code

```
import java.util.Scanner;

public class JavaBasics {

    // Method to check Prime Number
    static boolean isPrime(int n) {
        if (n <= 1)
            return false;

        for (int i = 2; i <= n / 2; i++) {
            if (n % i == 0)
                return false;
        }

        return true;
    }

    // Method to check Palindrome
    static boolean isPalindrome(int n) {
        int original = n;
        int reverse = 0;

        while (n != 0) {
            int digit = n % 10;
            reverse = reverse * 10 + digit;
            n = n / 10;
        }
        return original == reverse;
    }
}
```

```

    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Part 1: Command Line Arguments
        System.out.println("===== COMMAND LINE ARGUMENTS =====");

        if (args.length == 0) {
            System.out.println("No command line arguments provided.");
        } else {
            for (int i = 0; i < args.length; i++) {
                System.out.println("Argument " + (i + 1) + ": " + args[i]);
            }
        }

        // Part 2: Prime Number
        System.out.println("\n===== PRIME NUMBER =====");

        System.out.print("Enter a number: ");
        int number = sc.nextInt();

        if (isPrime(number))
            System.out.println(number + " is a Prime Number.");
        else
            System.out.println(number + " is not a Prime Number.");

        // Part 3: Palindrome
        System.out.println("\n===== PALINDROME =====");

        System.out.print("Enter a number: ");
        int palindromeNumber = sc.nextInt();

        if (isPalindrome(palindromeNumber))
            System.out.println(palindromeNumber + " is a Palindrome.");
        else
            System.out.println(palindromeNumber + " is not a Palindrome.");

        sc.close();
    }
}

```

■ Compilation & Execution Commands

Save the source code exactly as:

```
JavaBasics.java
```

Compile the program:

```
javac JavaBasics.java
```

Run the compiled program with command-line arguments:

```
java JavaBasics Hello CSM
```

Then give the following inputs: **17** (prime check) and **121** (palindrome check).

■ Output Screen

```
===== COMMAND LINE ARGUMENTS =====
```

```
Argument 1: Hello
```

```
Argument 2: CSM
```

```
===== PRIME NUMBER =====
```

```
Enter a number: 17
```

```
17 is a Prime Number.
```

```
===== PALINDROME =====
```

```
Enter a number: 121
```

```
121 is a Palindrome.
```

```

2. Even or Odd
3. Largest of Three Numbers
4. Grade Calculation
5. Day of Week
6. Multiplication Table
7. Numbers from 1 to N
8. Sum of First N Natural Numbers
9. Fibonacci Series
10. Factorial
11. Prime Number
12. Maximum of Two Numbers
13. Area of Circle
0. Exit
Enter your choice: 1
Enter Student Name: togi dinkar
Enter Roll Number: 259
Enter Marks in Subject 1: 30
Enter Marks in Subject 2: 50
Enter Marks in Subject 3: 60
|
----- Student Details -----
Student Name : togi dinkar
Roll Number : 259
Total Marks : 140
Percentage : 46.67%

===== STUDENT UTILITY PROGRAM =====
1. Student Information
2. Even or Odd
3. Largest of Three Numbers
4. Grade Calculation
5. Day of Week
6. Multiplication Table
7. Numbers from 1 to N
8. Sum of First N Natural Numbers
9. Fibonacci Series
10. Factorial
11. Prime Number
12. Maximum of Two Numbers

```

Program Execution Screenshot

■ Result

Thus, the Java program was successfully executed to demonstrate command-line arguments, prime number checking, and palindrome number checking using methods.